

Assignment 2: UAV Vertical Motion and PID Control

Name: Thomas Rompré

Student ID: B990389904

Date: 5/5/2026

1. Introduction

As mentioned in the handout, this assignment involves the simulation and analysis of open-loop thrust and feedback control in the context of controlling the vertical motion of a UAV.

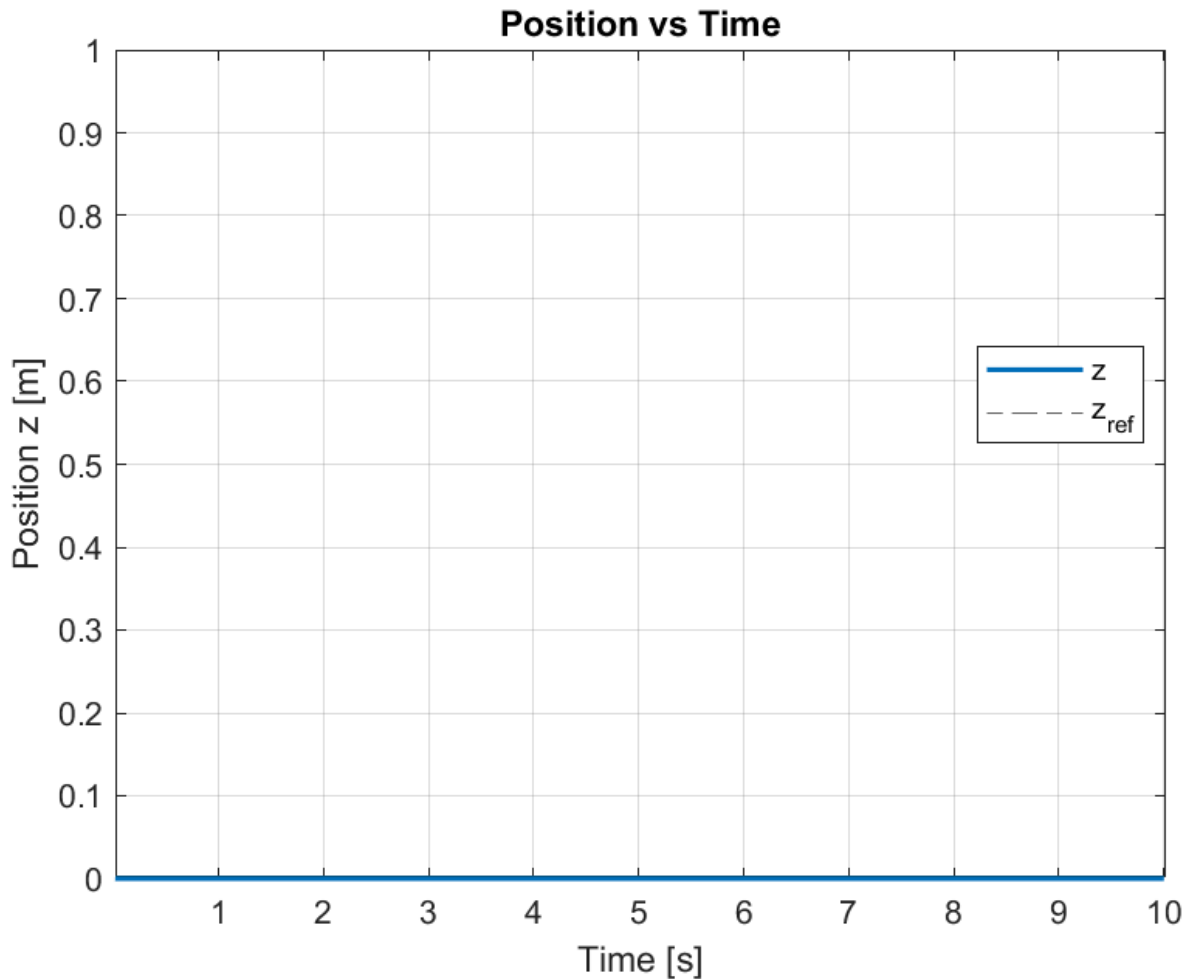
2. Step 1: Hovering Condition

Setup

- Parameters
 - $T = mg$
 - $K_p = 0, K_d = 0, K_i = 0$

Result

Figure 1: Altitude vs Time (Hovering Condition)



Observations

The altitude does not change over the span of the 10 seconds of simulations. This is mainly because the thrust is of equal value of mass * gravity. Meaning the upwards force of the thrust is equal to the downward "pull" of gravity and the UAV's own weight, these two equal forces cause the UAV to "hover" in place.

Explanation

In the case of $m\ddot{z} = T - mg$ T and mg are equal to each other, meaning that the result will always be 0. $\ddot{z}$ being the acceleration (moreso the vertical movement of the UAV in this sample / cycle). This means that the vertical position will not change.

3. Step 2: Open-Loop Drift

Setup

Test:

- Case A: $T > mg$
- Case B: $T < mg$

Result

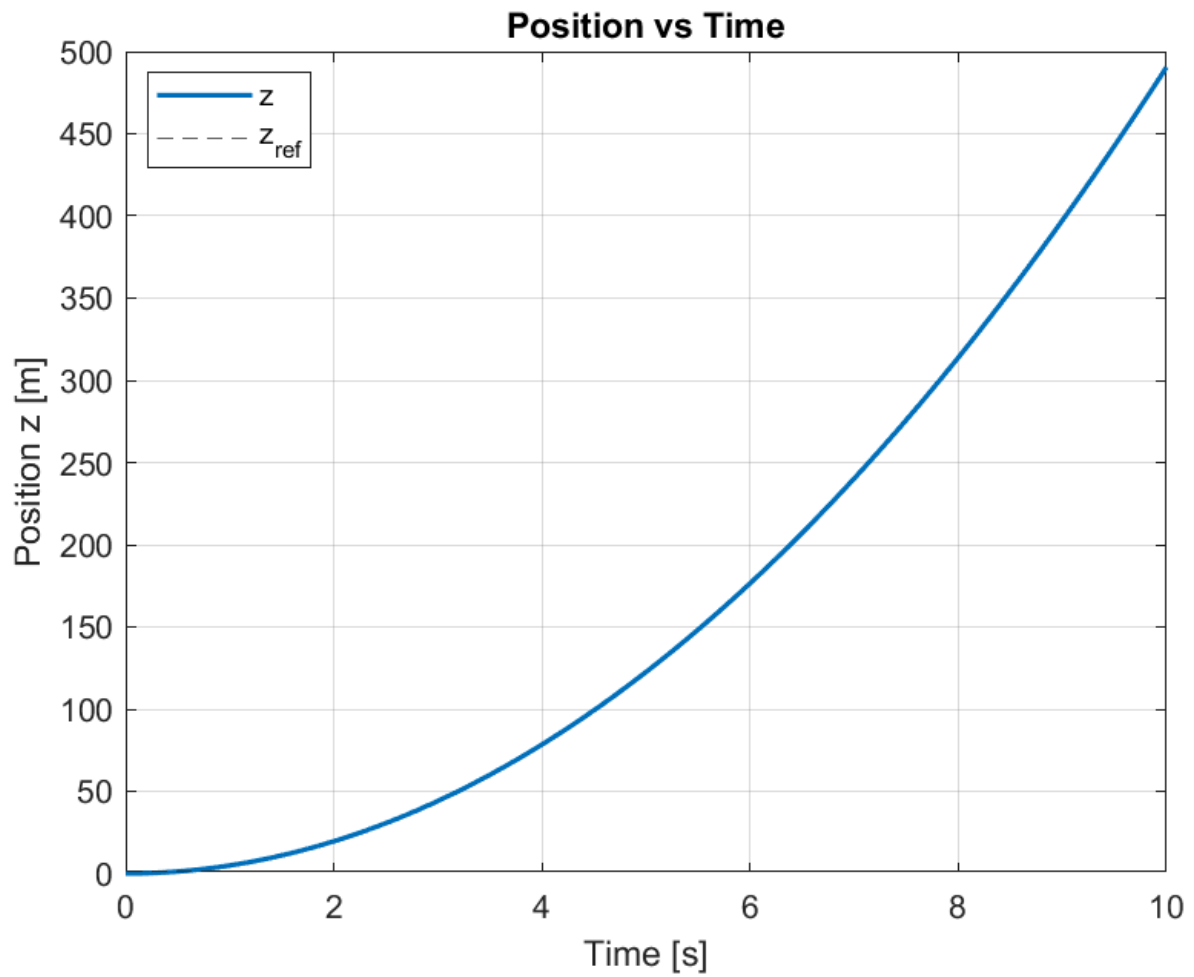


Figure 2: Altitude vs Time (Open-loop, $T > mg$)

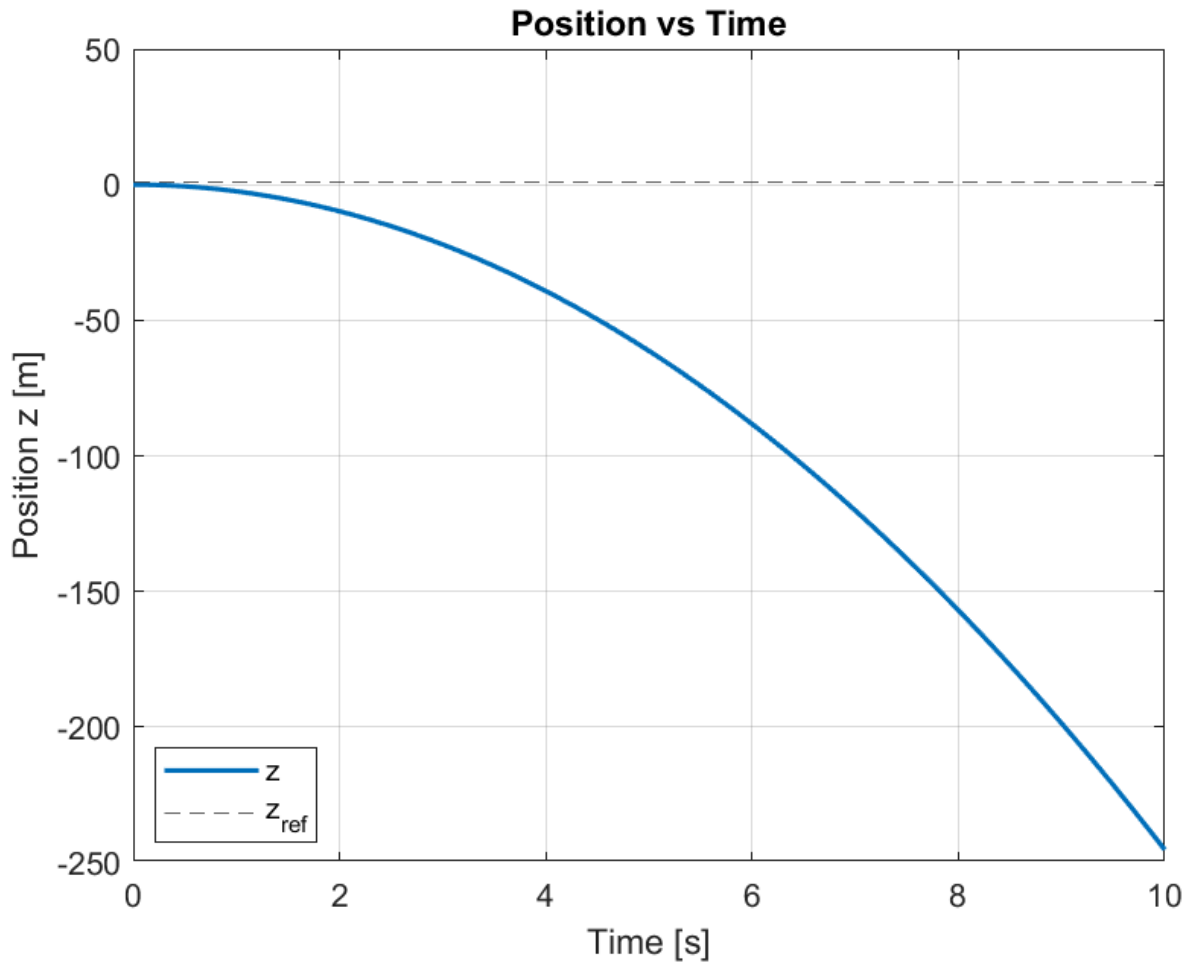


Figure 3: Altitude vs Time (Open-loop, $T < mg$)

Observations

When T is greater than mg , the UAV rises, when the opposite is true, the UAV falls.

Explanation

When movement occurs, it also carries with it inertia, which is factored into the acceleration. This causes a constant movement in a specific direction to increase exponentially if continuous with little to no opposing force (gravity or thrust, depending on the case).

4. Step 3: Proportional Control

Setup

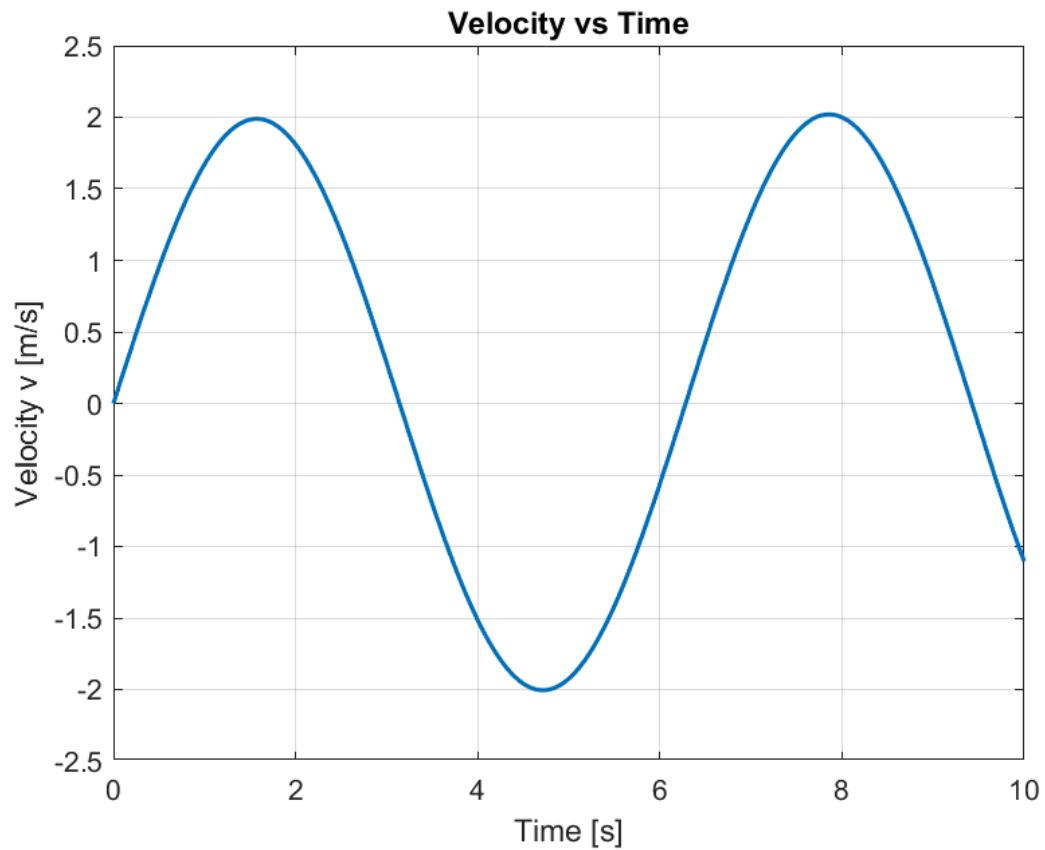
Control law:

$$T = \alpha mg + K_p(z_{ref} - z)$$

- $\alpha = 1.1$
- Test two values of K_p :
 - $K_p = [value\ 1]$
 - $K_p = [value\ 2]$

Result

Figure 4: Altitude vs Time (P control)



Observations

With a small K_p of 1.0:

- We see an oscillation between two absolute equal points of 2, looping roughly every 6.1 seconds.

With a larger K_p of 5.0:

- The same oscillation at a slightly greater magnitude of roughly 2.75, at a faster rate of roughly one loop per 2.8 seconds.

The sinusoidal movement is similar in both cases, and the speed at which we “loop” between our maximum and minimum altitude and the difference between these two altitudes increase when we increase K_p , the speed increasing at a seemingly faster rate than the altitude. This is because K_p takes into account the difference between the current position and the target position, without accounting for other factors, primarily inertia.

5. Step 4: Derivative Action (PD)

Setup

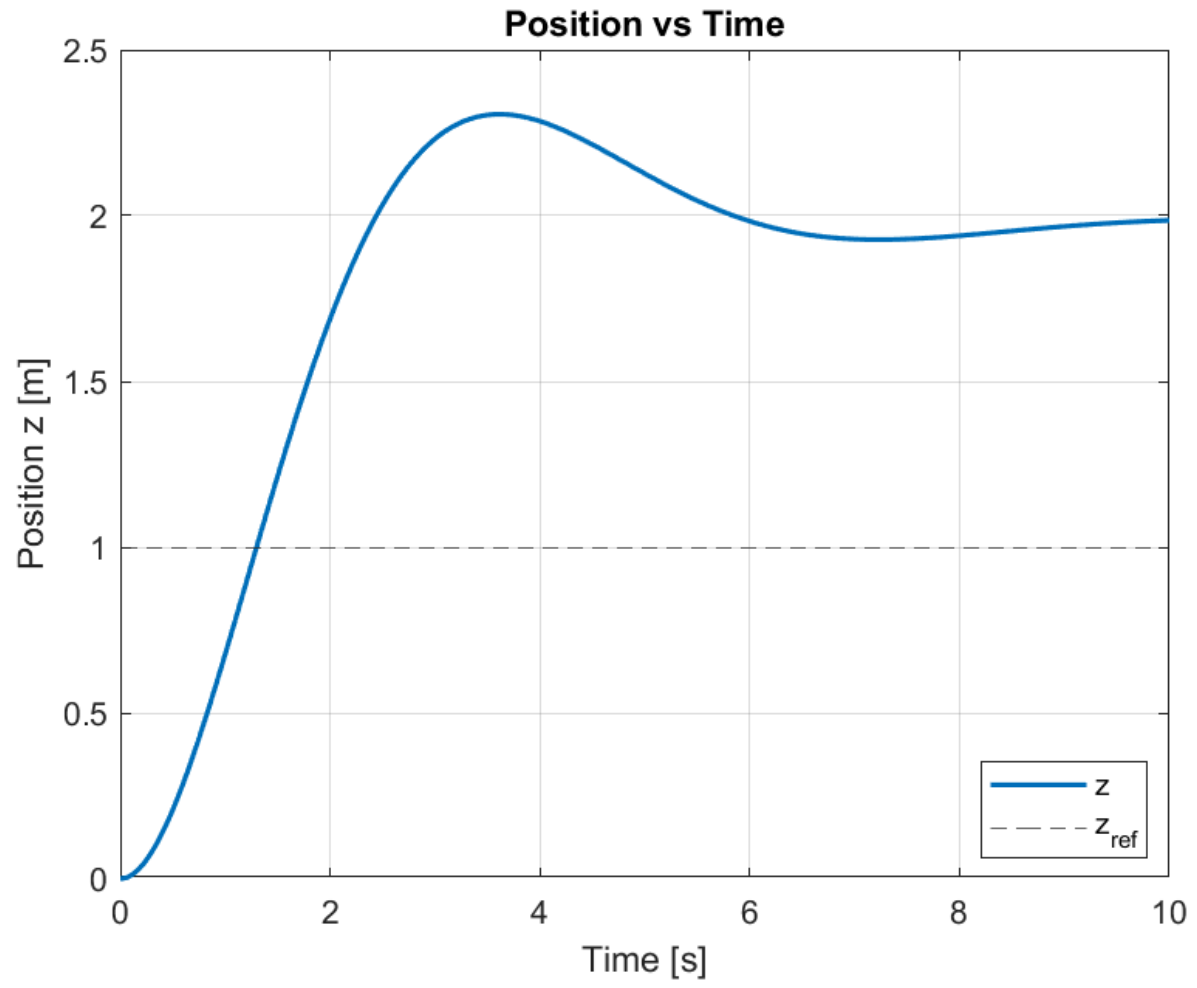
Control law:

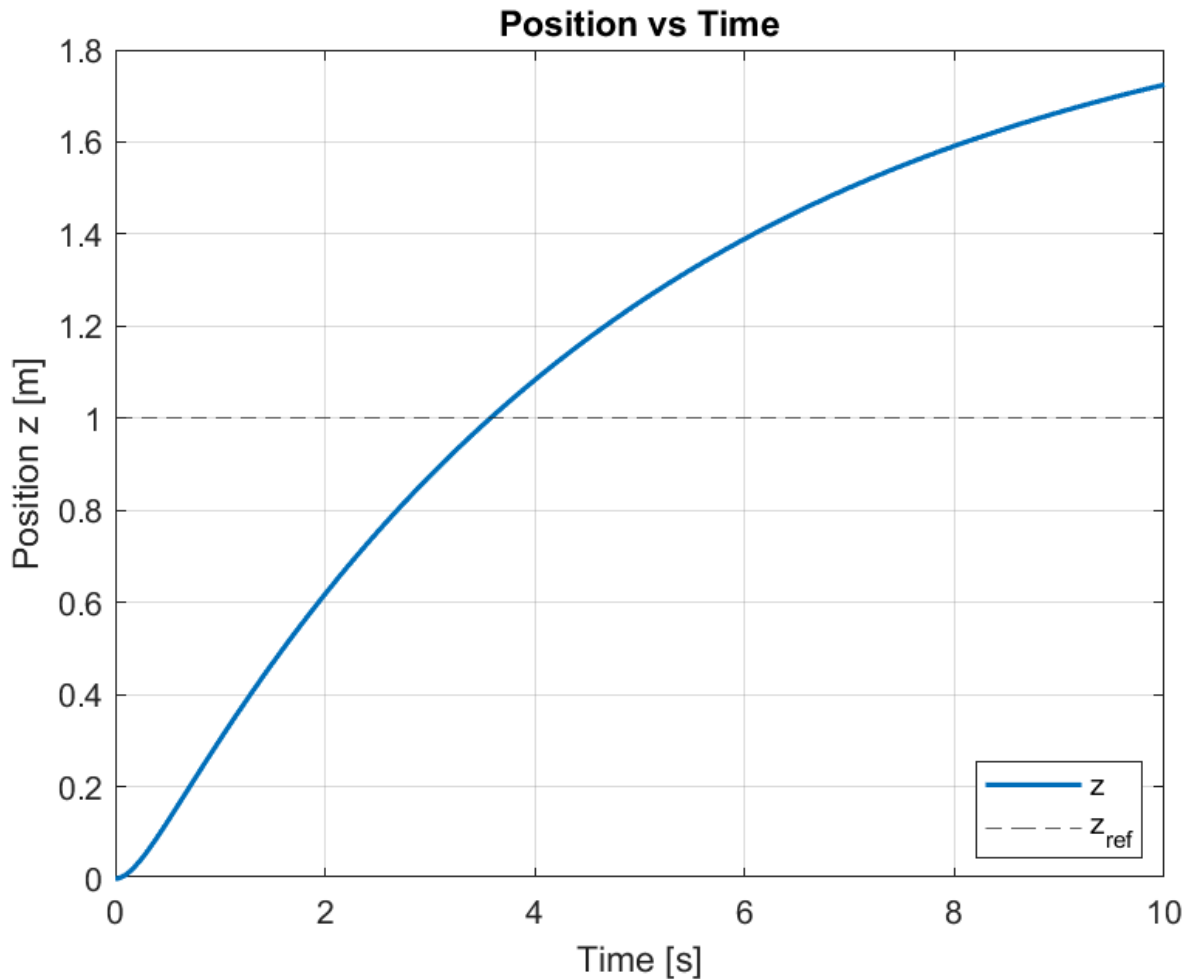
$$T = \alpha mg + K_p(z_{ref} - z) + K_d(v_{ref} - v)$$

- $v_{ref} = 0$
- $\alpha = 1.1$
- Test two values of K_d :
 - $K_d = [\text{value 1}]$
 - $K_d = [\text{value 2}]$

Result

Figure 5: Altitude vs Time (PD control)





Observations

With a small K_d :

- The UAV seems to slow down the closer it gets to the target position (z_{ref}), I assume by reducing the amount of thrust to ideally balance it out with the mg the closer we get to z_{ref} in order to achieve a 'hover' state while at that point.
- This is necessary as arriving too fast would cause the inertia to make the UAV move *past* the target position, and require it to move back downwards by reducing thrust, thus, by slowing down we remove a lot of the inertia to try and achieve that hover state as closely as possible.
- Higher K_d (1.0 on first plot, 5.0 on second) makes the 'deceleration' occur much earlier and at a much higher rate, it technically takes longer to reach

the target position of 2.0 (which is strange because it is set to 1.0 on the params but whatever) but it achieves a balanced hover state more accurately than with a Kd of 1, which initially rises past the Zref point and requires a downward movement to return to that position, with potentially too much inertia causing it to move past it again and repeat the process.

- This achieves a higher level of accuracy than modifying Kp which only reacts to current position without taking in account for acceleration or other factors, but can also take a longer time to achieve a desired position when comparing it to Kp.

6. Step 5: Integral Action (PID)

Setup

$$e = z_{ref} - z$$

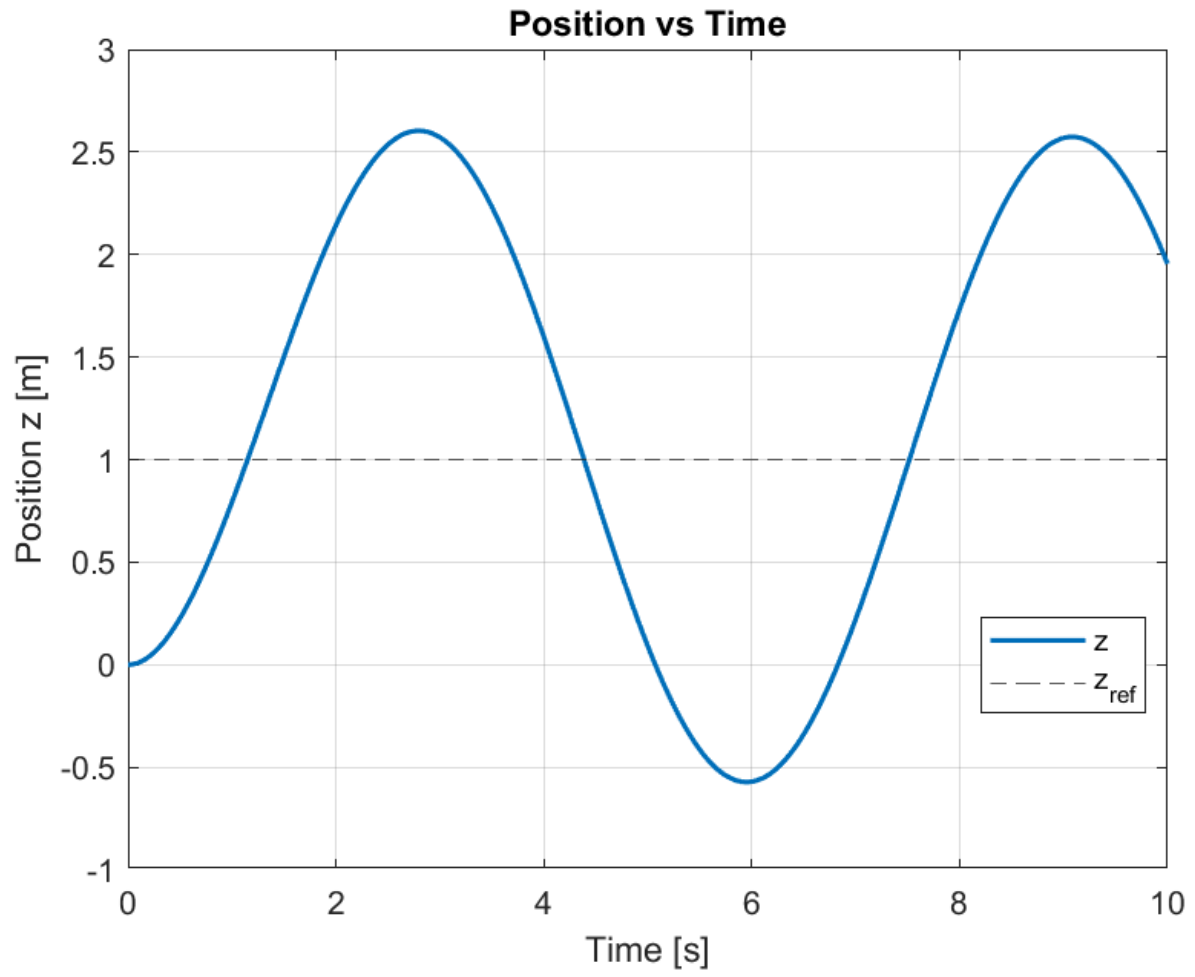
$$e_{int} = e_{int} + e \cdot T_s$$

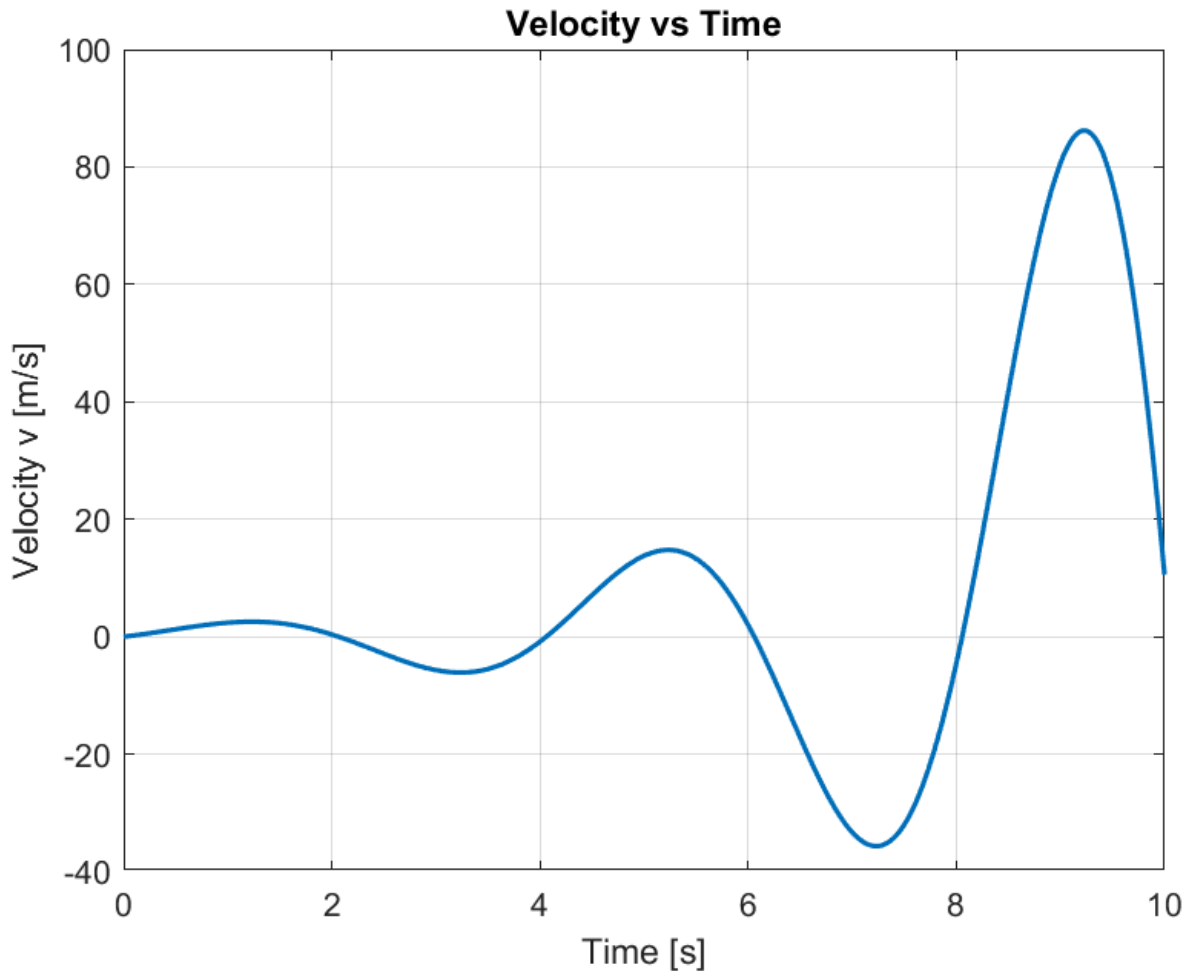
$$T = \alpha mg + K_p e + K_i e_{int} + K_d (v_{ref} - v)$$

- $\alpha = 1.1$
- Test two values of K_i :
 - $K_i = [value\ 1]$
 - $K_i = [value\ 2]$

Result

Figure 6: Altitude vs Time (PID Control)





Observations

Modifying [p.ki](#) causes the drone to exponentially fuck up over time. At a low value of 1.0, it oscillates wildly, at a higher value of 5.0, it begins to oscillate around its initial position before oscillating wildly.

A loose interpretation of code comments reveals that Ki relies on summer error derivatives (accumulated difference between current position and target position). By that logic, it would be safe to assume that if your Ki value is erroneous when it comes to calibrating your PiD, then it becomes *exponentially* erroneous.

7. Conclusion

The main objective of the PiD loop is to try and close the distance between its current position and the target position as much as possible. These calculations

determine how much the UAV should modify its current thrust in order to achieve this objective. Depending on which variable you modify and how you wish to calculate it, the method and effects are different.

- With P, we only take into account the difference between position and target position in order to calculate the necessary thrust to reach it.
- With D, we also take into account the current acceleration or inertia of the UAV to calculate the thrust in an effort to not overshoot.
- With I, we calculate the sum of the difference between position and target in order to calculate thrust.

Mess with these three variables until you get the desired results in terms of speed, accuracy and stability.